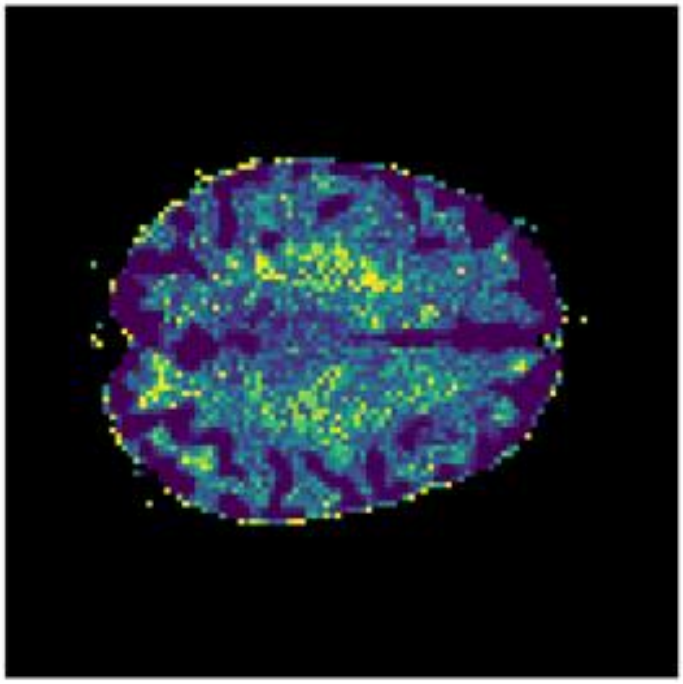
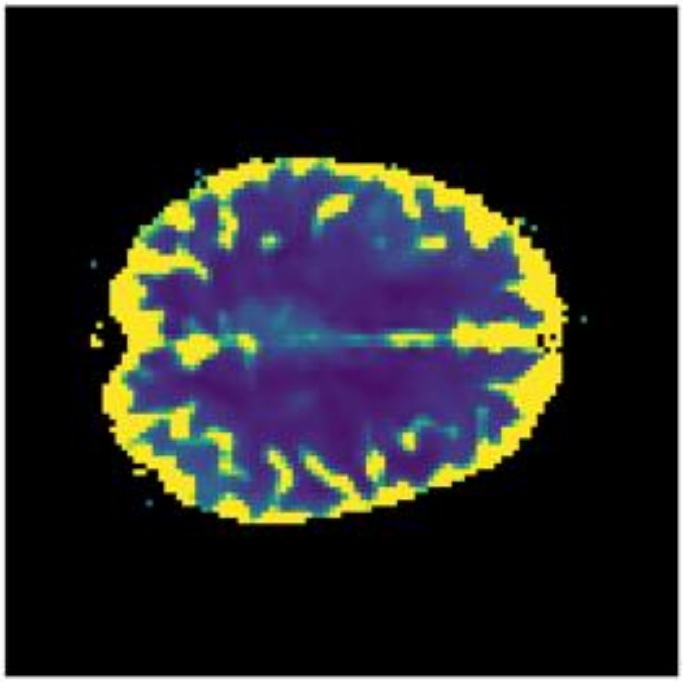
Project JIMM

Global search approach using a Patricle Swarm Optimizer (PSO) to calculate MWF values from MRI invivo relaxation data

- Python Routine [03.2024] -

Author: Martin Kobe, UFZ Leipzig, 2024
E-Mail: martin.kobe@ufz.de; martin.kobe@email.de





Inhalt

1. Requirements and preparations 3
   1.1. UFZ internal Windows-based high-performance computing systems 3
   1.2. Prepare your Python environment (examplarily by using Anaconda prompt) 3
   1.3. Create your lokal working environment and get Python routines 4
   1.4. Directory and program structure overview 5
2. Particle swarm optimization – PSO – how the software works 6
   2.1. To-do-list for Users: Input area in PSOmain.py 6
      2.1.1. Invivo data sources 6
      2.1.2. PSO class initialization variables 6
      2.1.3. Mode of PSO execution 9
   2.2. To do List for Users: PSOparameters.py 10
      2.2.1. Class Inversion() 10
      2.2.2. Class PSO() 10
      2.2.3. Class T1(), T2(), T2S() 10
   2.3. Run PSO routine and plot the results 11
      2.3.1. Results: Created directories and files 11
      2.3.2. Results: Python objects stored in local variables 13
      2.3.3. Plots: Vizualization ideas for publication 14
3. Troubleshooting and known bugs 17

1. Requirements and preparations

1.1. UFZ internal Windows-based high-performance computing systems

(a) Model Server Grid (MSG, recommended) or Model Server Grid Special2

- MSG: 4*112 Cores (actually about 60 of them are usable for a User), 767 GB RAM

  → High and fast performance on parallelized applications
- Recommended Python distribution: Anaconda 3 (Anaconda Navigator or prompt)
- Python package installation: pip (prompt) or Anaconda Navigator (GUI)

  NOTE: conda commands are currently not available on MSG (missing OpenSSL certificate)
- MSG Special2: not recommended before maintenance (NoSpaceLeftError)

(b) Datascience1 or Datascience2

- 72 cores and 6 TB RAM

  → High and fast performance on RAM-intensive applications
- Recommended Python distribution: Anaconda 3 (Anaconda Navigator or prompt)
- Python package installation: conda/pip (prompt) or Anaconda Navigator (GUI)

More information & links: https://www.intranet.ufz.de/index.php?de=44753

Linux-based systems: https://www.intranet.ufz.de/index.php?de=44757

1.2. Prepare your Python environment (examplarily by using Anaconda prompt)

- conda create --name ***yourenv*** python=3.x ***yourIDE***

yourenv:	choose name for your environment
yourIDE:	choose your IDE, recommended is spyder
Python version (optional):	for example 3.9, 3.10, 3.11

- conda activate ***yourenv***
- pip install ***required_package***

  NOTE: Packages required from online libraries are
  matplotlib, natsort, nibabel, pymp, scipy, cython

- pip install ***git_package***

  NOTE: Packages required from git repositories (follow README.md)
  pyepg - https://github.com/JIMM-MRI/epg
  epyg - https://github.com/JIMM-MRI/EpyG

  NOTE for trouble shooting:
  (a) MSG still needs C++ update for installing **pyepg** with pip
  (b) Alternatively, copy all files and dirs from directory 'PSOJIMM**\git_epg_pyepg\cp_3x\'** in the site-packages of your newly created environment. **cp_3x** means your version of Python!
  (c) For example on MSG copy into:
  C:\Users\yourname\.conda\envs***yourenv***\Lib\site-packages\

1.3. Create your lokal working environment and get Python routines

Shown examplarily by using Anaconda prompt:

- cd /D S:\
  → Change into the scratch-directory (1 TB free space for each User)

- mkdir ".\MWF invivo\Python"
  → Create destination folder for your PSO project

- option 1:
  git clone -b dev https://github.com/JIMM-MRI/mwf-invivo.git ".\MWF invivo\Python"
  → Clone the git repository using URL
  → Cloning installs PSO and site-scripts from the developer branch "dev"

- option 2:
  git clone -b dev git@github.com:JIMM-MRI/mwf-invivo.git ".\MWF invivo\Python"
  → Clone the git repository using SSH
  → Cloning installs PSO and site-scripts from the developer branch "dev"

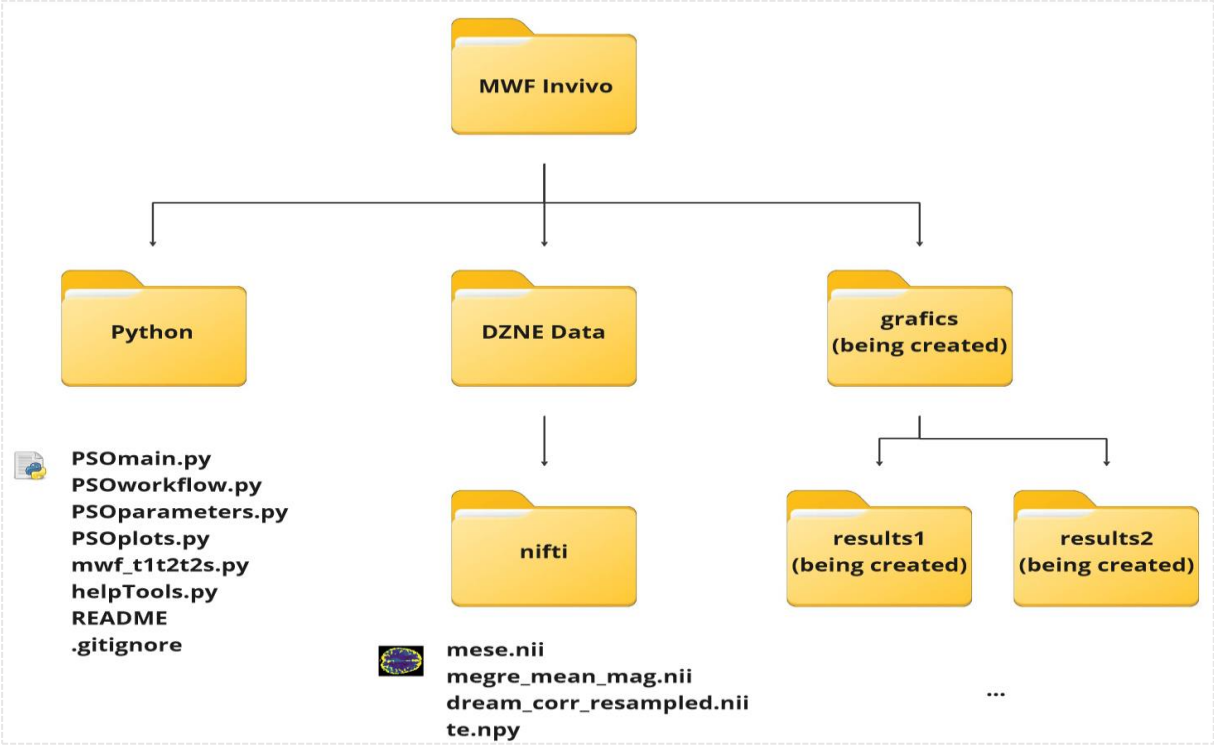
- option 3:
  As alternative to using git bash download ZIP-package manually from from https://github.com/JIMM-MRI/mwf-invivo/tree/dev and extract it into your working space.

- (Alternative to git-repository:
  Create the workspace by extracting MWF invivo from zipped PSOJIMM.zip
  → **NOTE:** Python scripts in here are not beeing updated or supported as git-repository

- mkdir ".\MWF invivo\DZNE Data\nifti"
  → Finally, create a new directory and store MRI relaxation T1, T2, T2S data files in here
  → 2023 data you will find in the directory **PSOJIMM in the UFZ cloud**

1.4. Directory and program structure overview

- Ensure that the dir-structure outlined above is in place after completing steps 1.1.-1.3.
  NOTE: The directory "grafics" and some subdirectories are created during PSO execution



- **Filelist Python scripts for PSO on MRI invivo data:**

PSOmain:	main routine with parallelization on multiple GPU cores
PSOworkflow:	class containing methods for PSO workflow and related calculations
PSOparameters:	parameter intervalls for setting up particle swarm in the model room
PSOplots:	class containing different methods for visualization of PSO results
mwf_t1t2t2s:	methods for MWF analysis on MRI data (Tony Stoecker, DZNE Bonn)
helpTools:	useful tool box for data handling; methods for Gaussian integration
README.md:	metafile with author's comments
.gitignore:	git metafile that specifies intentionally untracked files to ignore

- **Filelist MRI invivo data (file names are required and can be adjusted in PSOmain):**

mese:	T2 signal file - multi echo spin echo
megre_mean_mag:	T2S signal file - multi echo gradient echo
dream_corr_resampled:	map of scaling factors (nominal flip angles of the sequences
te.npy:	array of measurement time steps of T2S signal

2. Particle swarm optimization – PSO – how the software works

2.1. To-do-list for Users: Input area in PSOmain.py

In the main script, you'll find a section at the beginning of the file where you need to define and specify certain input variables. Additionally, there are comments provided for each variable that will guide you through the user input sections, and they are self-explanatory. Despite this, some 'unclear' parameters will still be described and explained below. Please maintain the structure and formatting to ensure a bug-free performance of the particle swarm optimizer main script.

2.1.1. Invivo data sources

```
# data and save path
NII_dataPath  = 'S:/JIMM/invivoData/DZNE_data/nifti'
NII_savePath  = 'S:/JIMM/invivoData/grafics'
NII_keyString = None
# invivo data files
T1_File       = 'NO_FILE'                  # (not measured in the 2023 session)
T2_File       = 'mese.nii'                 # multi echo spin echo
T2S_File      = 'megre_mean_mag.nii'       # multi echo gradient echo
T2SP_File     = 'megre_mean_mag.nii'       # phase information of T2S
B1_File       = 'dream_corr_resampled.nii' # radiofrequency (RF) transmit field
T2S_TE_File   = 'te.npy'                   # 1D-array of TE times
# additional specifications
dataType      = 'meas'      # 'meas' for invivo data, 'synt' for synthetic data
addNoise      = (False, 50) # tuple: (boolean, relatively added noice 1/SNR)
useMask       = True        # mask valid for slice18, derived from previous JI
pathMask      = f'{NII_dataPath}/maskSlice18.npy'  # location of your mask file
noSlice       = 18          # specify the slice number to be analysed
```

T2SP_File placeholder for now (!!!)
NOTE: - if not set, a not known error occurs (troubleshoot with Tony Stoecker)
- required as variable **KW_T2SP** for calling mwf_t1t2t2s.MWF_analysis()

dataType please keep 'meas' for now, as synthetic data atlas has different dimensions

useMask from previous MWF maps by JI calculation I derived a mask map in order to reduce execution time of the PSO algorithm; valid for slice 18 for now; mask using np.nan

2.1.2. PSO class initialization variables

```
# PSO algorithm initialization
noIter        = 100      # number of iteration while executing PSO
noPart        = 400      # number of particles for executing PSO
noPSOIterSli  = 25       # number of cycles of single PSO executions on a slice
noPSOIterPix  = 1000     # number of cycles of single PSO executions on a pixel
# mathematical specifications
lpNorm        = 'L2'     # LP-NORM --> L1-Norm: Manhattan & L2-Norm: Euclidean
invOpt        = 'V0'     # NUMBER of model vectors --> V0: 2 | V1: 1
noPeaks       = 'DIRAC'  # GAUSS: m1 (MW), m2 (FW) | DIRAC: m1 (MW), m2 (AW), m3 (EW)
```

NOTE: Above-variables turned to be a good compromise in terms of execution time and performance.

lpNorm: LP-Norm for normalization of the objectfunktion ('L0' or 'L1')
NOTE: 'L1' is not fully implemented yet

invOpt: number of model vectors for joint inversion ('V0' or 'V1')
NOTE: 'V1' is not fully implemented yet

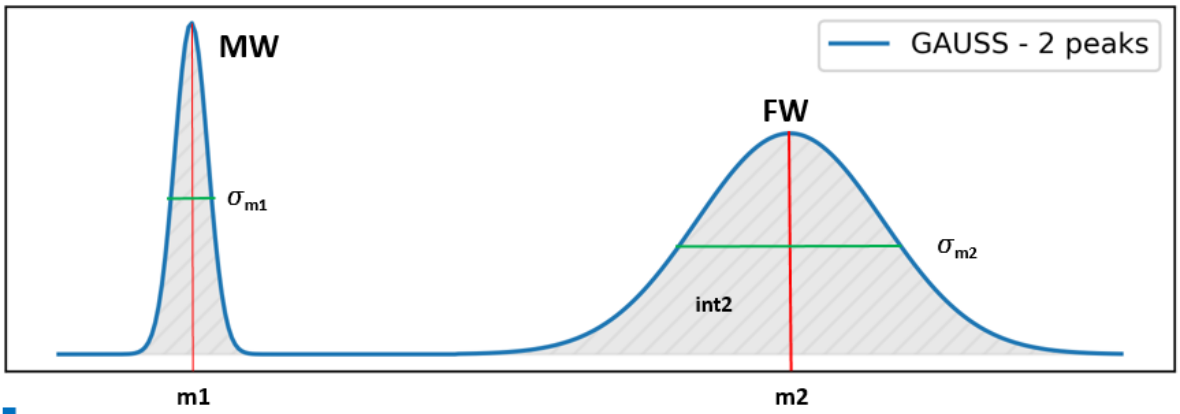
noPeaks: number of peaks used for integration in the model room ('GAUSS' or 'DIRAC')

Examples (shown for a Joint Inversion of 2 signals)

(i) invOpt='V0' + noPeaks='GAUSS'

Synthetic data will be calculated by intergration over 2 Gaussian bell curves and by using one model vector for each signal, respectively. The MWF parameter is assumed to the same for both signals.

NOTE:
The intervals in which the model parameters, listed in the model vectors below, can occur, can be defined in the parameter class stored in PSOparameters.py (cf. section 2.2.c).



$$\begin{bmatrix} m1_{sig1} \\ \sigma_{m1sig1} \\ m2_{sig1} \\ \sigma_{m2sig1} \\ int2_{sig1} \\ MWF \end{bmatrix} \quad \begin{bmatrix} m1_{sig2} \\ \sigma_{m1sig2} \\ m2_{sig2} \\ \sigma_{m2sig2} \\ int2_{sig2} \\ MWF \end{bmatrix}$$

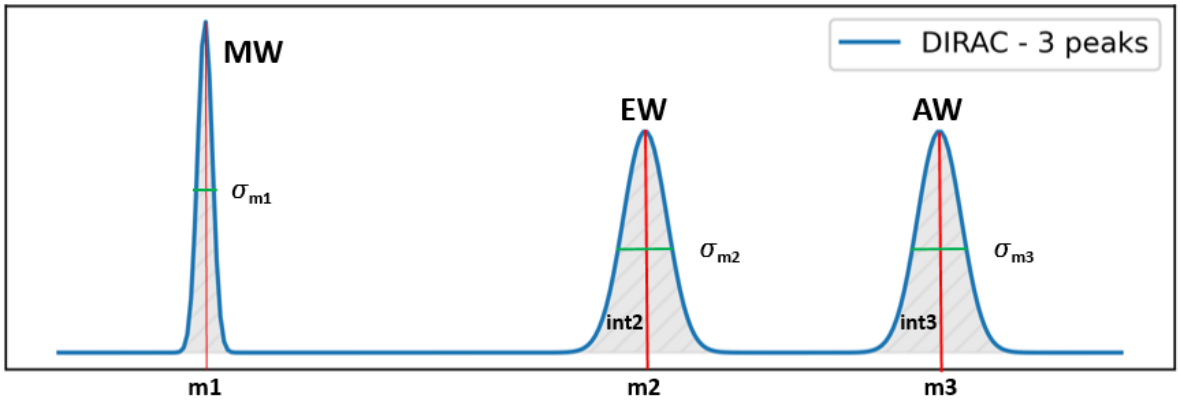
m*n*	-	n^{th} component mean value
σ_{mn}	-	n^{th} component SD
int*n*	-	area beyond the nth gaussian
MWF	-	Myelin Water Fraction
MW	-	Myelin Water
FW	-	Free Water
EW	-	Extracelluar Water
AW	-	Axonal Water

(ii) invOpt='V0' + noPeaks='DIRAC'

Synthetic data will be calculated by intergration over 3 Gaussian bell curves represented as Dirac delta functions (or a mix of both) by using one model vector for each signal, respectively. The MWF parameter is assumed to the same for both signals.

NOTE #1:
The intervals in which the model parameters, listed in the model vectors below, can occur, can be defined in the parameter class stored in PSOparameters.py (cf. section 2.2.c).

NOTE #2:
It appears that with the current physical model approach, all noise is being "dumped" into parameter m3. Discussion with Tony and Hendrik is required. Compare section 2.3.3.b



$$\begin{bmatrix} m1_{sig1} \\ \sigma_{m1sig1} \\ m2_{sig1} \\ \sigma_{m2sig1} \\ m3_{sig1} \\ \sigma_{m3sig1} \\ int2_{sig1} \\ int3_{sig1} \\ MWF \end{bmatrix} \quad \begin{bmatrix} m1_{sig2} \\ \sigma_{m1sig2} \\ m2_{sig2} \\ \sigma_{m2sig2} \\ m3_{sig2} \\ \sigma_{m3sig2} \\ int2_{sig2} \\ int3_{sig2} \\ MWF \end{bmatrix}$$

m*n*	-	n^{th} component mean value
σ_{mn}	-	n^{th} component SD
int*n*	-	area beyond the nth gaussian
MWF	-	Myelin Water Fraction
MW	-	Myelin Water
FW	-	Free Water
EW	-	Extracelluar Water
AW	-	Axonal Water

2.1.3. Mode of PSO execution

Within the main routine in PSOmain.py, you can choose the mode you want to execute as follows. This is facilitated through a set of booleans. In total, there are several possible blocks between which you can choose or execute all at once. **NOTE:** Parameters explained in the following text are bolded**.**

```
61    # mode of PSO application on invivo data
62    # (booleans) --> choose signal type and inversion type
63    invT1, invT2, invT2S = False, True, True
64    singleInv, jointInv  = False, True
65    # (a) iteration test
66    noIterTest    = False    # find reasonable iteration numbers (only for single inv)
67    percentThresh = 0.5      # associated threshold in percent
68    # (b) parallelized PSO
69    calcPSO_slice = False    # parallelized PSO over all pixels in a slice
70    calcPSO_pixel = False    # parallelized PSO over one pixel with noPSOIterPix iterations
71    pixelList     = [(45,50), (45,55), (37,60), (40,30)] # associated list of pixels
72    resultsList   = ['*dirpath*/T2T2S_results.npy']      # list with a result file (slice)
73    # (c) post hoc visualization
74    plotResults   = True        # generally plot results from a .npy-file
75    plotSlice     = True        # visualize MWF map of a slice from PSO results
76    plotPixelAcc  = True        # visualize accuracy of a pixel from PSO results
77    valOutliers   = (None, 0.1) # cut outliers in acc.plot, tuple: (lower,upper)
78    valPercentile = (0, 50)     # cut percentile in acc.plot, tuple: (lower,upper)
79    resultsDir    = [f'{NII_savePath}/100I400P25PSO/T2T2S/GAUSS/'] # list with a result dir
```

(booleans)

The example above specifies the execution of a Joint Inversion with T2 and T2S MRI signals.

(a) iteration test

If True, a PSO application is executed to distinguish of how many iterations and particles (cf. sec. 2.1.b) converging the bestfit solution? Test here is: fit(n)-fit(n-1) < **percentThresh**. **NOTE:** This option is only usable for the definition singleInv=True.

(b) Parallelized PSO

As the name says, here you can define if you want to execute the global search approach on a complete slice of a measured data set (**calcPSO_slice=True/False**) or on just one pixel of a single slice (**calcPSO_pixel=True/False**). Performing a high number of PSO cycles on the some object enables an accuracy measure for the MWF calculation. As the PSO execution on a complete slice is time consuming (about an hour for one cycle of Joint Inversion on MSG), it makes sense to run the PSO on single pixels with a strongly increased number of cycles (cf. sec. 2.1.b). The calculations will be performed parallelized on a maximum number of available cores of the used grid. Three blocks are defined in the main script (see PSOmain.py starting @ line around 340):

(i)	Block b.1:	CalcPSO_slice == True
(ii)	Block b.2:	CalcPSO_slice == True and calcPSO_pixel == True
(iii)	Block b.3:	CalcPSO_slice == False and calcPSO_pixel == True

NOTE #1:
Block b.3 is useful if you calculated the Myelin Water Fraction for a whole slice before and subsequently want to calculate and visualize the accuracies for a list of pixels. Here, you need to define a **pixelList** and also the associated MWF map for a slice in a **resultsList**.

NOTE #2:
By default all results and parametrizations will be visualized and stored as described in section 2.3.

(c) Post hoc visualization

In case you want to visualize your results, that are stored in .npy-files, again with different labels, headings, titles, informations, arrangements, or post processed results of accuracy point clouds, then set **plotResults=True**. For now, two different suggestions for result visualization (**plotSlice and plotPixelAcc**) are implemented. Plot methods and strings can be adjusted or added in the script PSOplots.py.

NOTE: The parameters **valOutliers** and **valPercentile** are described in detail in the plot section 2.3.3.

2.2. To do List for Users: PSOparameters.py

2.2.1. Class Inversion()

Contains general parameters such as model room size, and specific parameters for the preparation of signal models and calculation of systems matrices. These are used for the calculation of synthetic MRI relaxation curves.

2.2.2. Class PSO()

Contains literature-based values for the coefficients w - inertia weight factor, c1 - social weight factor, c2 - cognitive weight factor in the velocity update equation:

$$vel = w * vel * c1 * r1 * (bestPos - Pos) + c2 * r2 * (globBestPos - Pos)$$

2.2.3. Class T1(), T2(), T2S()

In order to initialize a particle swarm to approximate a bestfit-solution via optimization, the possible occurrence intervall for each model parameter shown in section 2.1.b has to to be predefined.

NOTE: The model parameter intervalls of each PSO execution will be stored in a parameter .json-file.

```python
    class T2S():

        def __init__(self):

            self.GAUSS = self.GAUSS()
            self.DIRAC = self.DIRAC()

        class GAUSS():

            def __init__(self):
                self.noPara = 6           # model vector size
                self.m1     = (5,   35)   # center of the gaussian, 1st peak
                self.m1_sig = (0.1, 0.1)  # standard deviation of m1
                self.m2     = (51,  130)  # center of the gaussian, 2nd peak
                self.m2_sig = (0.1, 5)    # standard deviation of m2
                self.int2   = (0,   5)    # area under the curve of m2 gaussian
                self.MWF    = (0,   0.85) # typical intervall of MWF in a MRI

        class DIRAC():

            def __init__(self):
                self.noPara = 9           # model vector size
                self.m1     = (5,   25)   # center of the gaussian, 1st peak
                self.m1_sig = (0.1, 0.1)  # standard deviation of m1
                self.m2     = (30,  80)   # center of the gaussian, 2nd peak
                self.m2_sig = (0.1, 10)  # standard deviation of m2
                self.m3     = (70,  130)   # center of the gaussian, 3rd peak
                self.m3_sig = (0.1, 40)  # standard deviation of m2
                self.int2   = (0.1, 5)    # area under the curve of m2 gaussian
                self.int3   = (0.1, 5)    # area under the curve of m3 gaussian
                self.MWF    = (0,   0.85) # typical intervall of MWF in a MRI
```

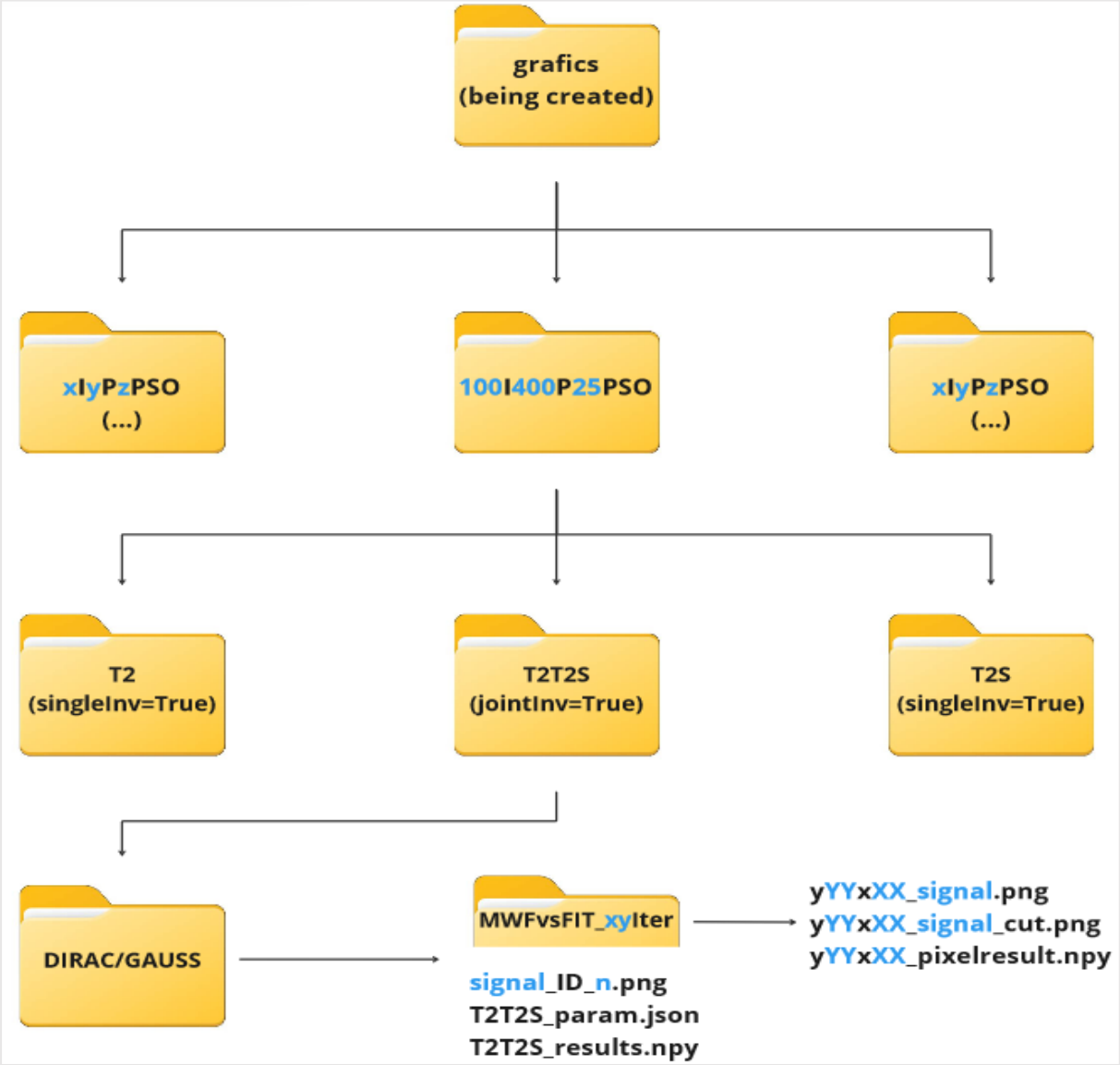
The model parameter intervalls are exemplarily shown for MRI signal T2S. Through adept adjustments if the **sigmas**, Gaussian bell curves can be manipulated into various forms (such as camel humps, delta peaks, etc) to closely approximate reality. Also care, that areas under the curve (**int2 and int3**) are being realistic, as these parameters turned to have a big influence on the final MWF results.

2.3. Run PSO routine and plot the results

Once you have set and specified all necessary variables and model parameters run PSOmain.py either in your API or by typing Python PSOmain.py into the (anaconda) console/prompt assuming you are in the projects main directory as defined in section 1.3.

2.3.1. Results: Created directories and files

After running of the PSO for the first time you will find the following structure of directories and files created in your working environment. Exemplarily shown for a joint inversion of the MRI signals T2 and T2S.



NOTE: Blue marked letters in the above grafic mark variables which depend on the pre-specifications of PSO algorithm variables (cd. section 2.1.b) and on the type of inverted signal.

File list generated during the execution of the PSO main routine

T2T2S_param.json:	dictionary of predefined PSO specifications and model parameters
T2T2S_results.npy:	dictionary of numpy arrays with PSO results for a whole slice
yYYxXX_pixelresult.npy:	dictionary of numpy arrays with PSO results for a pixel
signal_ID_n.png:	PSO result visualization of a slice by PSOplots.plotSlice()
yYYxXX_signal.png:	PSO accuracy visualization of a pixel by PSOplots.plotMWFvsFIT()
yYYxXX_signal_cut.png:	PSO accuracy visualization of a pixel by PSOplots.plotMWFvsFIT()

2.3.2. Results: Python objects stored in local variables

(a) result_map_sl (also saved into *_results.npy)

- dictionary of numpy arrays with PSO results for a whole slice
- numpy array shape is [n, 110, 110, **m**]
  → n=7 (GAUSS) and n=10 (DIRAC)
  → **m=noPSOIterSL+1** and therefore stores specific results for each PSO cycle
  → **NOTE:** numpy array stores best global fit values for model parameters in [n, 110, 110, -1]

n (Python starts with index 0)	GAUSS	DIRAC
0	m1	m1
1	σ_{m1}	σ_{m1}
2	m2	m2
3	σ_{m2}	σ_{m2}
4	int2	m3
5	MWF	σ_{m3}
6	fitness value	int2
7		int3
8		MWF
9		fitness value

Example:

→ Best global MWF map for signal T2 after Joint Inversion of T2 and T2S with 3 peaks (DIRAC):

```
data      =  np.load('*yourdir*/ T2T2S_results.npy', allow_pickle=True).item()
globMWF   =  data['T2'][8, :, :, -1]
```

(b) result_map_px (also saved in *_pixelresult.npy)

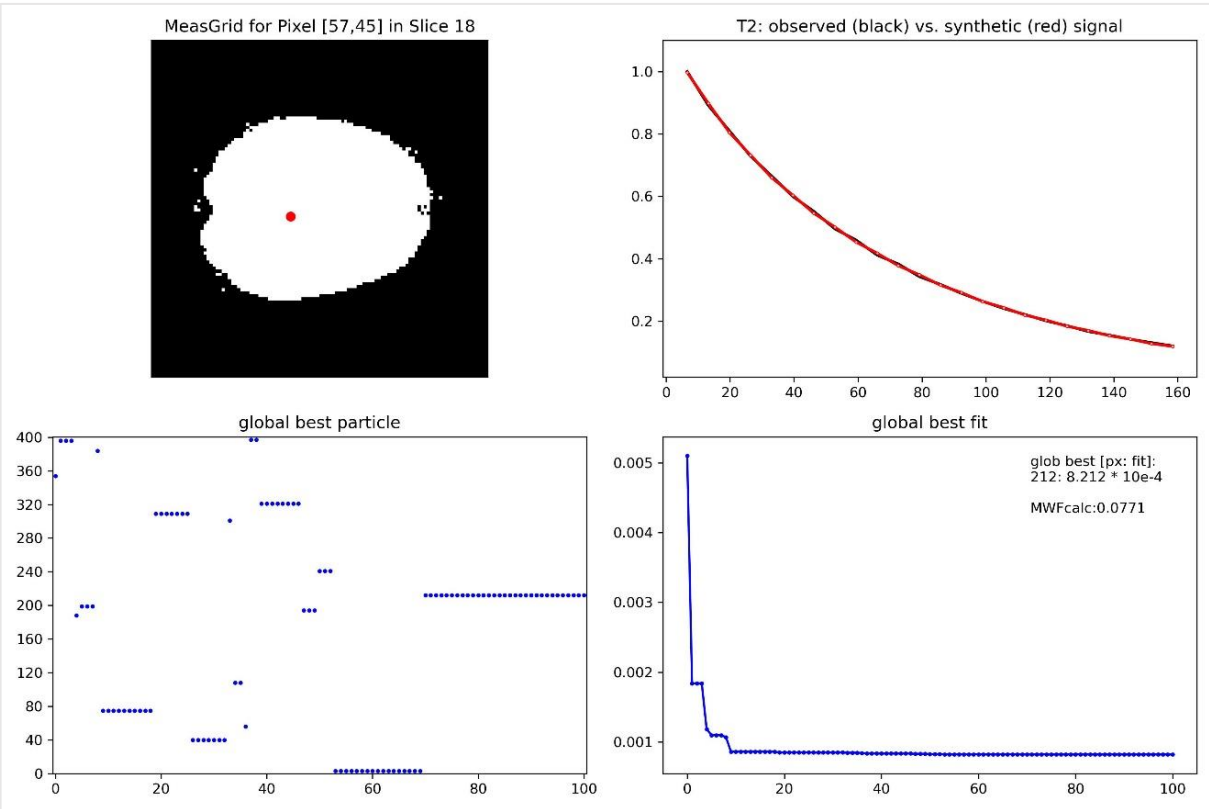
- dictionary of numpy arrays with PSO results for a pixel
- numpy array shape is [n, **m**]
  → n=7 (GAUSS) and n=10 (DIRAC)
  → m=noPSOIterPix
  → **NOTE:** content stored in n is the same as described in the table above!

2.3.3. Plots: Vizualization ideas for publication

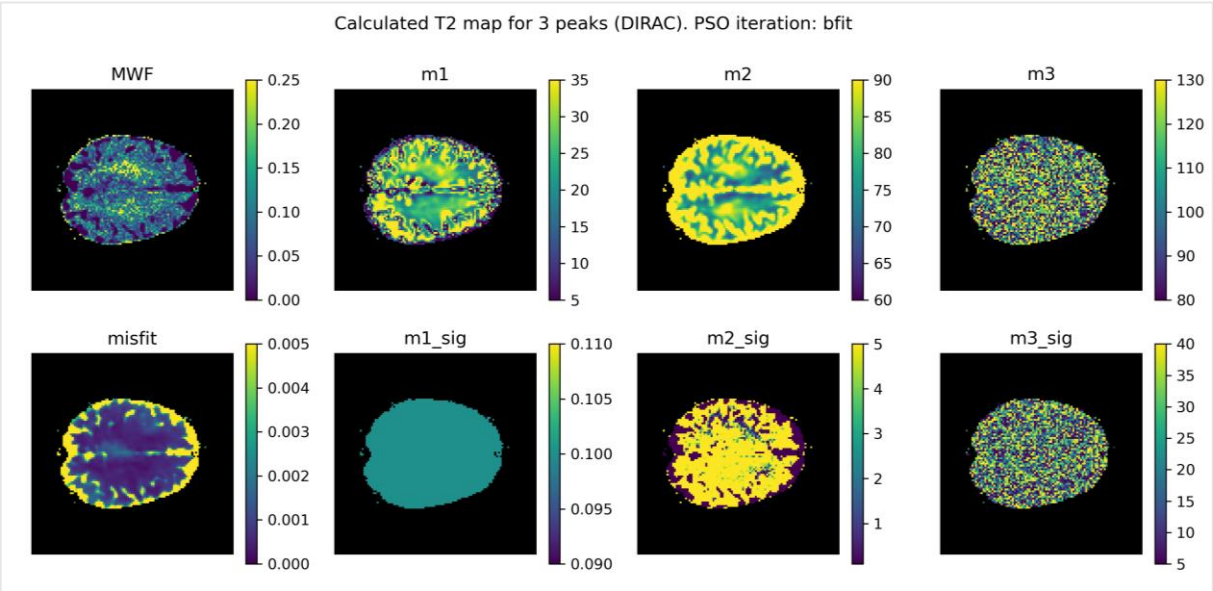
Result visualization is done by default while executing PSOmain algorithm. Also, plot methods, labels, strings, pre processings, etc., can be either adjusted or added in the script PSOplots.py. In the following several visualization ideas are introduced as brainstorming for future work and publication.

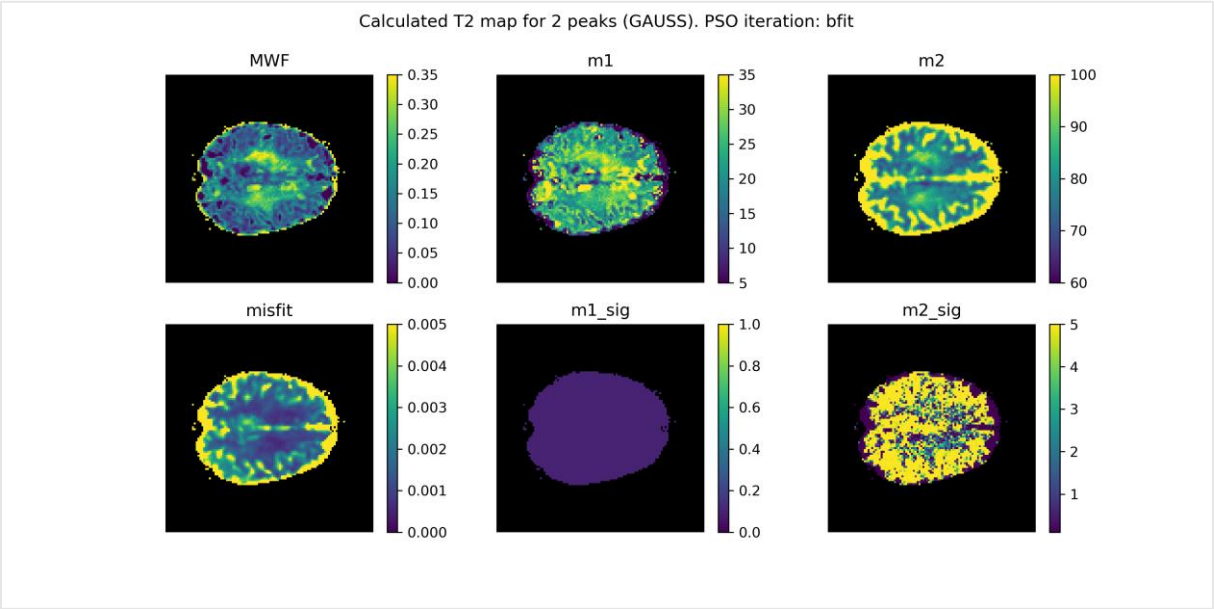
a. plotIterTest()

- enabled if **noIterTest==True**
- by default calculated for a number of pixels in the very center of a slice
- this type of visualization is useful for determining the optimal combination of iteration count and pixel count to achieve the best possible PSO performance when executing MWF mapping for a slice or a pixel (calcPSO)



b. plotSlice()

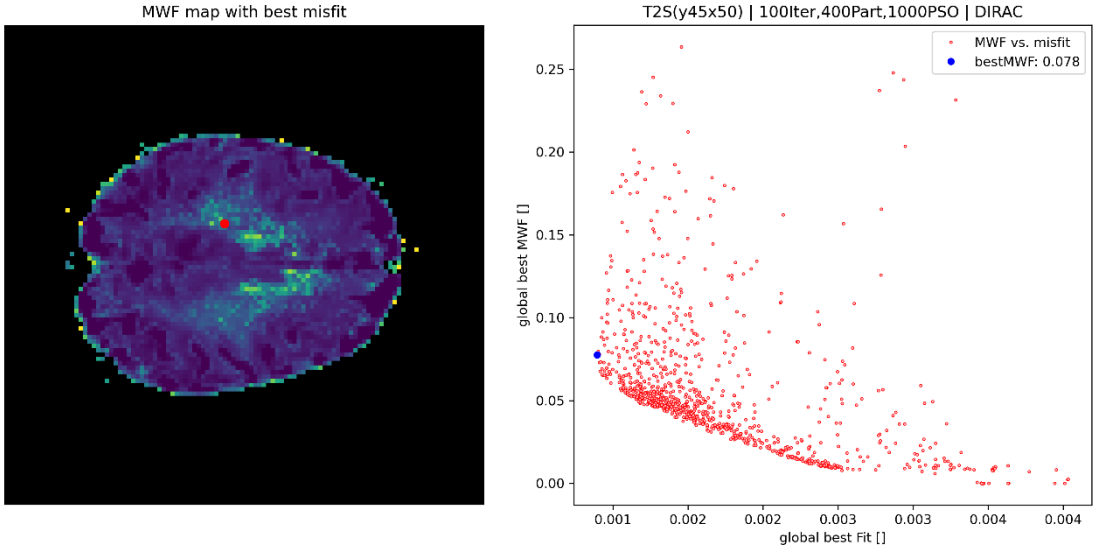




- colorbar intervalls depend on model parameter intervalls in PSOparameters.py or if chosen, on the savings in the *.json parameter file
  NOTE: MWF and misfit is chosen manually based on experience.
  → interval boundaries can be adjusted in the plot function using the parameter 'lim'

- DIRAC approach: all noise is being "dumped" into parameter m3
- it may also be interesting to plot the area under the curve, i.e. model param int2 or int3

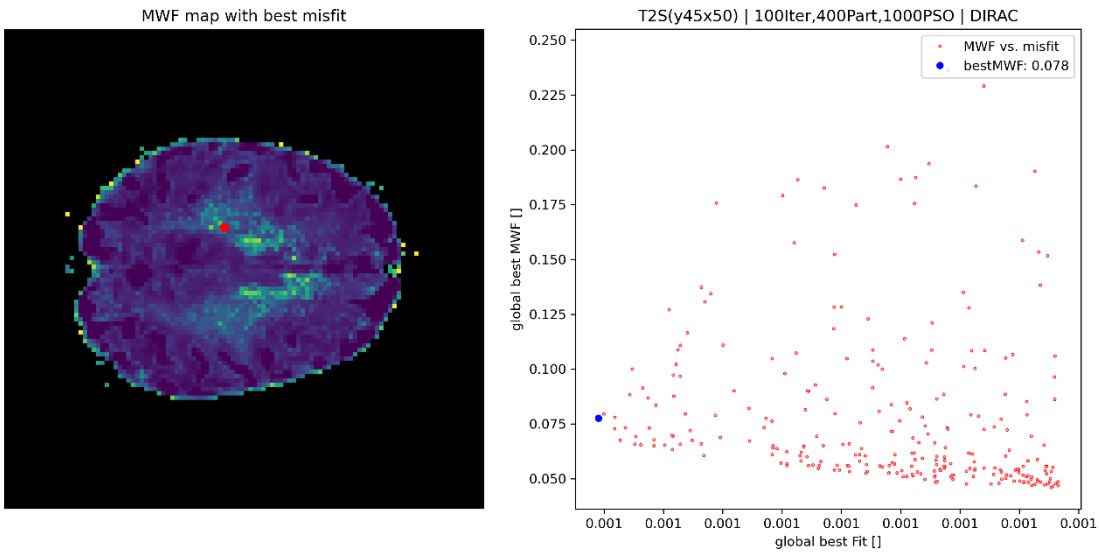
c. plotMWFvsFIT()



- comparison between global best MWF and global best fit for a high number of PSO executions on one pixel as a point cloud; the pixel coordinates are shown as red dot within the global best MWF map for a given model parameter constellation
- such a kind of grafic could serve as an (in)accuracy measure for the global search performance and calculated final best model parameter constellation

- in the 2nd grafic (below) the point cloud is reduced by:
  (a) a cut for outliers (point cloud contains all values that fit: median+stdv***valOutliers[1]**; compare section 2.1.3.)
  (b) a cut of points that are outside of lower (**valPercentile[0]**) and upper (**valPercentile[1]** percentile value

  NOTE: If you don't want to cut the point cloud, set the relevant parameter slots as False!



3. Troubleshooting and known bugs

- If you run PSO for different intervall boundaroes of PSO model parameters (as specified in PSOparameters.py) the created directories (sec.2.3.1.) will be overritten. A temporary solution is to manually add a count index to the directory name after each PSOmain execution.

- The Windows based server grids available at UFZ Leipzig sometimes show memory or CPU usage overflow (touch 100% of available ressources). Then either the program traces back an error or just freezes. You can check on the following URL (UFZ intranet available) how the ressources on the grids are currently in use:

  https://grafana.web-intern.app.ufz.de/d/ixkVDVBnz/ueberblick-msg?orgId=2